

Install Required Libraries

```
pip install tensorflow keras-tuner matplotlib numpy
```

Import Required Libraries

```
import tensorflow as tf
import keras_tuner as kt
import numpy as np
import matplotlib.pyplot as plt
import time

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import SGD, Adam, RMSprop
```

Load and Preprocess MNIST Dataset

```
# Load Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize Images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Training Shape:", x_train.shape)
print("Testing Shape :", x_test.shape)
```

Task 1: Experiment with Hidden Layers and Neurons

Create Model Function

```
def create_model(hidden_layers=1, neurons=64):

    model = Sequential()

    model.add(Flatten(input_shape=(28, 28)))

    for _ in range(hidden_layers):
        model.add(Dense(neurons, activation="relu"))
```

```
model.add(Dense(10, activation="softmax"))
```

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

```
return model
```

Train Different Models

```
configs = [  
    (1, 32),  
    (1, 64),  
    (1, 128),  
    (2, 64),  
    (2, 128)  
]
```

```
results = []
```

```
for layers, neurons in configs:
```

```
    print(f"\nLayers = {layers}, Neurons = {neurons}")
```

```
    model = create_model(layers, neurons)
```

```
    history = model.fit(  
        x_train,  
        y_train,  
        validation_data=(x_test, y_test),  
        epochs=5,  
        verbose=0  
    )
```

```
    accuracy = history.history["val_accuracy"][-1]
```

```
    results.append((layers, neurons, accuracy))
```

```
    print("Validation Accuracy:", round(accuracy * 100, 2), "%")
```

Sample Results

Hidden Layers Neurons Validation Accuracy

1	32	97.20%
1	64	97.80%
1	128	98.10%
2	64	98.20%
2	128	98.40%

Observation

- Increasing neurons generally improves learning.
 - Two hidden layers often perform better than one.
 - More layers increase training time and may overfit.
-

Task 2: Compare Batch Size and Epochs

```
batch_sizes = [16, 32, 64]
```

```
epochs_list = [5, 10, 20]
```

```
for batch in batch_sizes:
```

```
    for epochs in epochs_list:
```

```
        model = create_model()
```

```
        start = time.time()
```

```
        history = model.fit(
            x_train,
            y_train,
            batch_size=batch,
            epochs=epochs,
            validation_data=(x_test, y_test),
            verbose=0
        )
```

```

end = time.time()

print("-----")
print("Batch Size :", batch)
print("Epochs   :", epochs)
print("Training Time:", round(end-start,2), "seconds")
print("Validation Accuracy:",
      round(history.history["val_accuracy"][-1]*100,2), "%")

```

Example Results

Batch Size Epochs Training Time Validation Accuracy

16	5	40 sec	98.0%
16	10	78 sec	98.3%
16	20	150 sec	98.5%
32	5	22 sec	97.9%
32	10	45 sec	98.2%
64	5	14 sec	97.7%
64	20	55 sec	98.3%

Observation

- Smaller batch sizes take longer but often achieve slightly better accuracy.
- More epochs improve learning until the model converges.

Task 3: Compare Optimizers

```

optimizers = {
    "SGD": SGD(),
    "Adam": Adam(),
    "RMSprop": RMSprop()
}

```

```

plt.figure(figsize=(10,6))

```

```

for name, optimizer in optimizers.items():

```

```

model = create_model()

model.compile(
    optimizer=optimizer,
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    x_train,
    y_train,
    validation_data=(x_test,y_test),
    epochs=10,
    verbose=0
)

plt.plot(
    history.history["val_loss"],
    label=name
)

plt.title("Validation Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

```

Optimizer Comparison

Optimizer Speed Accuracy

SGD	Slow	97–98%
Adam	Fast	98–99%
RMSprop	Fast	98–99%

Observation

- **Adam** converges the fastest.
- **RMSprop** performs similarly to Adam.
- **SGD** often needs more epochs and learning-rate tuning.

Task 4: Hyperparameter Tuning with KerasTuner

Build Model

```
def build_model(hp):  
  
    model = Sequential()  
  
    model.add(Flatten(input_shape=(28,28)))  
  
    units = hp.Int(  
        "units",  
        min_value=32,  
        max_value=256,  
        step=32  
    )  
  
    model.add(Dense(units, activation="relu"))  
  
    model.add(Dense(10, activation="softmax"))  
  
    model.compile(  
        optimizer="adam",  
        loss="sparse_categorical_crossentropy",  
        metrics=["accuracy"]  
    )  
  
    return model
```

Hyperband Search

```
tuner = kt.Hyperband(  
    build_model,  
    objective="val_accuracy",  
    max_epochs=5,  
    directory="mnist_tuner",  
    project_name="hyperparameter_search"  
)  
  
tuner.search(  
    x_train,
```

```

        y_train,
        validation_data=(x_test, y_test)
    )

    best_hp = tuner.get_best_hyperparameters(1)[0]

    print("Best Number of Neurons:", best_hp.get("units"))

```

Example Output

Best Number of Neurons: 128

Note: The best value may vary across runs because of random initialization and training variability.

Task 5: Compare Weight Initialization Methods

```

initializers = [
    "he_normal",
    "glorot_uniform",
    "random_normal"
]

for initializer in initializers:

    print("\nInitializer:", initializer)

    model = Sequential([
        Flatten(input_shape=(28,28)),

        Dense(
            128,
            activation="relu",
            kernel_initializer=initializer
        ),

        Dense(
            10,
            activation="softmax"
        )
    ])

```

```

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

start = time.time()

history = model.fit(
    x_train,
    y_train,
    validation_data=(x_test,y_test),
    epochs=5,
    verbose=0
)

end = time.time()

print("Training Time:",
      round(end-start,2),"seconds")

print("Validation Accuracy:",
      round(history.history["val_accuracy"][-1]*100,2), "%")

```

Example Results

Initializer	Training Speed	Validation Accuracy
he_normal	Fast	98.2%
glorot_uniform	Fast	98.0%
random_normal	Slower	97.5%